ACTIVITY PACK

Activity 7 - Break a Leaky Chatbot, Then Compare the Guarded One

Complete learner scenario, practice and security checklist

Course	AI Security for Autonomous AI Agents
Course code	TGS-2025060473
Version	4.0 | 25 August 2026

Tertiary Infotech Pte Ltd | UEN 20120096W

Readme

Activity 7 - Break a Leaky Chatbot, Then Compare the Guarded One

Purpose: Try to make an unsafe chatbot leak fictional private data, then see how a guarded version blocks the very same attacks.

> ■■ **This is a deliberately insecure TRAINING demo with FICTIONAL data.** The "leaky" bot is built to fail on purpose so you can see the risk. Never use these techniques on any real system.

What you need

- A web browser
- `leaky-chatbot/index.html` and `guarded-chatbot/index.html` in this folder
- A **training** API key (from your trainer)
- The attack sheet in this folder: `ATTACK-PROMPTS.md`

Evidence status: SIM - all data is fictional. All names, bookings and "secrets" are made up (for example, the fictional guest *Tan Ah Kow*).

> **API key safety:** Use only the **low-limit training key**. **Never** a production or personal key. It is stored **only in your browser** for this session. Clear it when done.

Steps

1. Open `leaky-chatbot/index.html` and enter the training API key.
2. Ask a **normal question** first (for example, "What are your check-in times?") to see it behave helpfully.
3. Now try the **attack prompts** from `ATTACK-PROMPTS.md` one by one. Note what fictional private data leaks - customer bookings, internal memos, an "admin password", a guest's phone number.
4. Open `guarded-chatbot/index.html` and enter the same training API key.
5. Send the **exact same prompts**. Watch the guardrails block or safely refuse them.
6. For 3 - 4 prompts, write down **what the leaky bot revealed** and **how the guarded bot responded**.

The four guardrail layers (why the guarded bot is safe)

1. **Retrieval filter** - the bot can only fetch data it is allowed to; private records are never pulled in.
2. **Input guard** - checks your message for attack patterns ("ignore your instructions", "print the password") before the model sees it.
3. **Hardened prompt** - the bot's own instructions firmly refuse to reveal secrets or personal data.
4. **Output guard** - scans the answer before it is shown, and blocks it if private data slipped through.

What you produce

- A short table: prompt -> what the leaky bot leaked -> how the guarded bot responded
- One sentence naming which guardrail layer stopped each attack

Reflect (Data Privacy / Job Impact / Ethical Concerns / Cyber Security)

- **Data Privacy:** If this were real, whose data just leaked, and what harm could follow?
- **Job Impact:** Who in a real team is responsible for testing a chatbot like this before launch?
- **Ethical Concerns:** Is it acceptable to ship the "leaky" version to save time? Why not?
- **Cyber Security:** Which single guardrail would you never skip, and why?